

Programowanie obiektowe

Zbigniew Marszalek

Programowanie obiektowe

1. Przedstaw podstawowe założenia paradygmatu obiektowego
2. Przedstaw różnice między programowaniem obiektowym a programowaniem strukturalnym.
3. Przedstaw budowę klasy i interfejsu.
4. Wyjaśnij pojęcie abstrakcji w programowaniu obiektowym.
5. Wyjaśnij pojęcie hermetyzacji w programowaniu obiektowym.
6. Wyjaśnij pojęcie polimorfizmu w programowaniu obiektowym.
7. Przedstaw zastosowanie dziedziczenia w programowaniu obiektowym.
8. Przedstaw zastosowania przeciążenia metod i operatorów w programowaniu obiektowym.

Paradygmat programowania

- **Paradygmat programowania** (ang. *programming paradigm*) — wzorzec programowania przedkładany w danym okresie rozwoju informatyki ponad inne lub ceniony w pewnych okolicznościach lub zastosowaniach.
- **Paradygmat programowania** definiuje sposób patrzenia programisty na sposób budowy i wykonywanie programu komputerowego. Przykładowo, w programowaniu obiektowym jest on traktowany jako zbiór współpracujących ze sobą obiektów, podczas gdy w programowaniu funkcyjnym definiujemy, co trzeba wykonać, a nie w jaki sposób.

Przedstaw podstawowe założenia paradygmatu obiektowego

- **Programowanie obiektowe** (ang. *object-oriented programming*, OOP) – paradygmat programowania, w którym programy definiuje się za pomocą danych zapisanych w obiektach i zachowanie się obiektów wyrażone za pomocą metod. Obiektowy program komputerowy wyrażony jest jako zbiór obiektów, komunikujących się pomiędzy sobą w celu wykonywania zadań.
- W programowaniu proceduralnym, dane i procedury nie są ze sobą bezpośrednio powiązane. Programowanie obiektowe ma zapewnić łatwe pisanie, konserwację i wielokrotne użycie programów lub ich fragmentów.
- Największym atutem programowania, projektowania oraz analizy obiektowej jest zgodność takiego podejścia z rzeczywistością.

Obiekty i klasy

- Klasa – definicje format danych oraz dostępne metody dla danego typu lub obiektu klasy. Mogą zawierać zagnieżdżone klasy oraz metody.
- Obiekty – instancje klas.
- **Instancja** (łac. *instantia*) – pojedyncze wystąpienie niezależnego kodu zgodnego z danym wzorcem.

```
package pl;
public class Pracownik {
    private int id;
    private String nazwisko;
    private double placaca;
    // Deklaracja pól klasy
    public Pracownik() { }
    //Przeciążenie konstruktora domyślnego
    public Pracownik(int id, String nazwisko, double placaca){
    //Konstruktor publiczny
        this.id = id;
        this.nazwisko = nazwisko;
        this.placaca = placaca;
    }
    public int getId(){
        return id;
    }
    public void setId(int id){
        this.id = id;
    }
}
```



```
public String getNazwisko() {
    return nazwisko;
}

public void setNazwisko(String nazwisko) {
    this.nazwisko = nazwisko;
}

public double getPlaca() {
    return placa;
}

public void setPlaca(double placa) {
    this.placa = placa;
}

@Override
public String toString() {
    //Przeciążenie metody ToString()
    String s = String.format
        ("Id: %d nazwisko: %s placa: %g", id, nazwisko, placa);
    return s;
}
}
```

```
package p1;  
public class P1 {  
    public static void main(String[] args) {  
        Pracownik pr = new Pracownik(1, "Kowalski", 2500.0);  
        System.out.println(pr);  
    }  
}
```

Wyjście - P1 (run) X



run:



Id: 1 nazwisko: Kowalski płaca: 2500,00



BUILD SUCCESSFUL (total time: 0 seconds)



Podstawowe założenia paradygmatu obiektowego

1. Abstrakcja
2. Hermetyzacja
3. Polimorfizm
4. Dziedziczenie

Koncepcja programowania obiektowego wywodzi się z Simuli 67, języka zaprojektowanego do zastosowań symulacyjnych, stworzonym przez Ole-Johana Dahla i Kristena Nygaarda z Norsk Regnesentral w Oslo.

Przedstaw różnice między programowaniem obiektowym a programowaniem strukturalnym

- **Programowanie strukturalne** – paradygmat programowania opierający się na podziale kodu źródłowego programu na procedury i hierarchicznie ułożone bloki z wykorzystaniem struktur kontrolnych w postaci instrukcji wyboru i pętli. Rozwijał się w opozycji do programowania wykorzystującego proste instrukcje warunkowe i skoki.
- Początki programowania strukturalnego przypadają na Lata 60. XX wieku - współtwórca E. Dijkstra
- Język programowania zgodny z paradygmatem programowania strukturalnego nazywa się językiem strukturalnym.

Struktury kontrolne

- Podstawowe struktury kontrolne jakie wymienia twierdzenie o programowaniu strukturalnym, z których możliwe jest zbudowanie dowolnego programu to:
- **Sekwencja** – wykonanie ciągu kolejnych instrukcji
- **Wybór** – w zależności od wartości predykatu wykonywana jest odpowiednia instrukcja. W językach programowania zazwyczaj reprezentowana przez słowa kluczowe if..then..else.
- **Iteracja** – wykonywanie instrukcji póki spełniony jest jakiś warunek. Reprezentowana w różnych wariantach jako pętle oznaczane między innymi przez: while, repeat, for lub do..until.
- Każda ze struktur kontrolnych może być też rozumiana jako pojedyncza instrukcja. Według pierwotnych założeń struktury kontrolne powinny posiadać jedno wejście i jedno wyjście.

Programowanie strukturalne

- Podprogramy pozwalają na wydzielenie pewnej grupy instrukcji i traktowania ich jako pojedynczej operacji, są dodatkowo mechanizmem abstrakcji.
- Bloki odpowiadają sekwencjom instrukcji, umożliwiając budowanie programu przez komponowanie struktur kontrolnych – w miejscu, w którym umieścimy blok z instrukcjami jest on traktowany jak pojedyncza instrukcja.
- Opublikowany w roku 1958 ALGOL 58 oraz jego dwa lata późniejsza, mocno rozwinięta wersja ALGOL 60, wprowadziły wiele kluczowych konstrukcji programistycznych, istotnych nie tylko z punktu widzenia programowania strukturalnego. Był to pierwszy język, w którym pojawił się concept instrukcji złożonych – bloków – służących do grupowania instrukcji. Blok jest traktowany jako pojedyncza instrukcja i możliwe jest użycie go w miejscu dla niej przeznaczonym.

Twierdzenie

- Jako fundament teorii programowania strukturalnego określa się twierdzenie o programowaniu strukturalnym (Bhömowi i Giuseppe Jacopiniemu). W swojej pracy pokazali oni, że dowolny program zawierający skoki (czyli dowolny graf przepływu), może zostać znormalizowany do odpowiadającego mu ustrukturyzowanego grafu - zbudowanego z trzech podstawowych struktur: Π , Δ oraz Ω , wyrażającym odpowiednio: sekwencję, wybór oraz iterację. Ponieważ grafy przepływu mogą wyrażać dowolne funkcje obliczalne, to oznacza, że możemy też takie wyrazić przy pomocy tych podstawowych konstrukcji.

Obsługa wyjątków

- Badacze spierają się co do użycia wyjątków w kontekście programowania strukturalnego, jako że łamią one zasadę pojedynczego wyjścia. Cytując wcześniejsze badania (1999-2004) i podając własne, Westley Weimer i George Necula stwierdzają, że problem z wyjątkami polega na tym, że „tworzą ukryte ścieżki przepływu sterowania, o których programistom ciężko wnioskować”. Pojawiają się propozycje by sprowadzić wyjątki do pojedynczego wyjścia, jak i opinie, by pozostawić pełną dowolność w ich użyciu, jako że reguła pojedynczego wyjścia powstała przed pojawieniem się i popularyzacją wyjątków, a opakowywanie wyjątków by uczynić zgodnymi z regułą jednego wyjścia dodaje zbędne poziomy zagnieżdżenia utrudniając zrozumienie programów.

Przedstaw budowę klasy i interfejsu

- **Klasa** – definicje format danych oraz dostępne metody dla danego typu lub obiektu klasy. Klasy mogą zawierać zagnieżdżone klasy oraz metody.
- **Obiekty** – instancje klas.
- **Instancja** (łac. *instantia*) – pojedyncze wystąpienie niezależnego kodu zgodnego z danym wzorcem.

Klasa to szablon, na podstawie którego konstruowane są obiekty. W języku Java wszystko bazuje na klasach. Poniższy przykład pokazuje konstrukcję klasy Pracownik.

Definicja klasy

Definicja klasy składa się z opcjonalnej listy atrybutów, opcjonalnego modyfikatora, słów kluczowego `class` albo słów kluczowych `static class` oraz opcjonalnej listy klas bazowych lub interfejsów wykorzystywanych do tworzenia danej klasy. Po liście klas bazowych zapisany zostaje kod klasy (ciało klasy), które zawiera między innymi właściwości, konstruktory i metody. Przedstawmy prostą klasę przechowującą informacje identyfikatora użytkownika, jego nazwisko i płaca. Kod definicji klasy i programu tworzącego na jej podstawie obiekty może wyglądać następująco:

W przypadku tworzenia obiektu klasy korzystamy z konstruktora wywoływanego za pomocą polecenia `new`. Wszystkie klasy w języku Java dziedziczą po wspólnej klasie bazowej `System.Object`. Dziedziczenie to ma charakter domyślny i niejawny dlatego nie wymaga dokonywania dodatkowych deklaracji przy tworzeniu klasy.

Programowanie zorientowane obiektowo (ang. *Object Oriented Programming OOP*) bazuje na budowie obiektów posiadających pewne cechy i konstrukcji metod dokonujących zmian w istniejących obiektach lub tworzących nowe obiekty. Konwencjonalne oprogramowanie strukturalne polegało na zaprojektowaniu funkcji działających w oparciu o algorytmy, które rozwiązują postawione zadanie. W programowaniu obiektowym podstawowym zagadnieniem jest zdefiniowanie sposobu przechowywania danych, a dopiero później poszukiwania algorytmów umożliwiających przekształcania danych w żądany sposób.

Kluczem do uzyskania dużej wydajności w OOP jest powiązanie każdego obiektu z metodami wykonującymi operacje na danym obiekcie. W szczególności, obiekt nie powinien bezpośrednio manipulować danymi należącymi do innego obiektu, ani też nie powinien zezwalać na manipulowanie innym obiektom na swoich danych. Całość komunikacji powinna odbywać się za pomocą zdefiniowanych metod. Ukrywanie danych jest podstawowym zagadnieniem programowania obiektowego i nosi nazwę enkapsulacji. Enkapsulacja jest powiązaniem danych pomiędzy sobą i sposobem korzystania z nich w ramach pojedynczego pakietu. Dzięki enkapsulacji danych zwiększamy elastyczność programu i zmniejszamy ryzyko powstania błędów przy projektowaniu aplikacji.

```
package pl;
public class Pracownik {
    private int id;
    private String nazwisko;
    private double placaca;
    // Deklaracja pól klasy
    public Pracownik() { }
    //Przeciążenie konstruktora domyślnego
    public Pracownik(int id, String nazwisko, double placaca){
    //Konstruktor publiczny
        this.id = id;
        this.nazwisko = nazwisko;
        this.placaca = placaca;
    }
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
}
```

```
public String getNazwisko() {
    return nazwisko;
}

public void setNazwisko(String nazwisko) {
    this.nazwisko = nazwisko;
}

public double getPlaca() {
    return placa;
}

public void setPlaca(double placa) {
    this.placa = placa;
}

@Override
public String toString() {
    //Przeciążenie metody ToString()
    String s = String.format
        ("Id: %d nazwisko: %s placa: %g", id, nazwisko, placa);
    return s;
}
}
```

```
package p1;  
public class P1 {  
    public static void main(String[] args) {  
        Pracownik pr = new Pracownik(1, "Kowalski", 2500.0);  
        System.out.println(pr);  
    }  
}
```

Wyjście - P1 (run) X



run:



Id: 1 nazwisko: Kowalski płaca: 2500,00



BUILD SUCCESSFUL (total time: 0 seconds)



Interfejsy

Interfejs definiuje klasę i jej elementy bez określania ich dokładnych implementacji. Interfejs określa, co klasa ma rozbić, ale nie wskazuje, w jaki ma to robić. Interfejsy są składowo podobne do klas, ale nie mogą w nim pojawiać zmienne składowe a metody deklaruje się bez podawania jakiegokolwiek kodu. W ten sposób określa się metody, które klasa musi zaimplementować, ale nie wskazuje się jak ma tego dokonać. Dowolna liczba klas może implementować dany interfejs. Dowolna klasa może implementować wiele interfejsów. Interfejs spełnia następujące trzy warunki:

- Definicja interfejsu – interfejs definiuje składowe klasy, takie jak metody i inne, jednak sam nie udostępnia implementacji tych elementów.
- Klasa implementująca interfejs – klasa implementująca pewien interfejs tworzy implementację wszystkich metod zdefiniowanych w tym interfejsie. Klasa może je implementować w dowolny sposób.
- Klienci – obiekty tworzące egzemplarze klasy implementującej interfejs znane są pod nazwą klientów. Klient wywołuje metody zdefiniowane w interfejsie i wykorzystuje ich implementację w określonej klasie.

Różnice między interfejsem a abstrakcyjną klasą bazową.

- Klasa abstrakcyjna może zawierać zaimplementowane metody abstrakcyjne (przynajmniej jedną), natomiast interfejs nie może zawierać żadnych implementacji metod.
- Klasa abstrakcyjna może zawierać konstruktory i destruktory, które nie mogą pojawiać się w interfejsie.
- Klasa może implementować wiele różnych interfejsów, ale dziedziczyć może tylko po jednej klasie bazowej.

Definicja interfejsu

Interfejs definiuje się podobnie jak klasę i ogólna postać takiej definicji:

```
dostęp interface nazwa {  
    typ zmienna-finalna1 = wartość;  
    //...  
    typ zmienna-finalnaN = wartość;  
    zwracany – typ nazwa-metody1(lista – parametrów);  
    //...  
    zwracany – typ nazwa-metodyM(lista – parametrów);  
}
```

Element *dostęp* jest typu *public* lub nie jest stosowany wcale. Gdy pominie się specyfikację dostępu, interfejs będzie dostępny, jedynie dla klas tego samego pakietu. Dla dostępu *public* interfejs będzie dostępny dla wszystkich innych klas.

Wewnątrz deklaracji interfejsu mogą pojawiać zmienne. Są one jednak niejawne ustawione na zmienne typu *final* i *static*, co oznacza, że nie mogą być zmienione przez implementującą go klasę i trzeba je zainicjować. Wszystkie metody i zmienne są niejawnie ustawione na typ *public*, jeśli sam interfejs został zadeklarowany z użyciem słowa kluczowego *public*.

Poniżej znajduje się przykład interfejsu. Deklaruje on interfejs, który zawiera metodę o nazwie `callBack()` przyjmującą jeden parametr typu `int`.

Definiowanie interfejsu

```
package p6;  
interface Callback {  
    void callBack(int param);  
}
```

```
package p6;  
public class Client implements Callback{  
    public void callBack(int p){  
        System.out.println("Wywołanie metody callBack() "+p);  
    }  
}
```

```
package p6;  
public class AnotherClient implements Callback {  
    public void callBack(int p){  
        System.out.println("Inna wersja metody callBack() " + p*p);  
    }  
    void nonIfaceMeth(){  
        System.out.println("klasa implementująca interface");  
        System.out.println("może zawierać także własne metody");  
    }  
}
```

```
package p6;
public class P6 {
    public static void main(String[] args) {
        Callback c = new Client();
        AnotherClient ob = new AnotherClient();
        c.callBack(42);
        c = ob;
        c.callBack(42);
    }
}
```

Wyjście - P6 (run) ×	
	run:
	Wywołanie metody callBack() 42
	Inna wersja metody callBack() 1764
	BUILD SUCCESSFUL (total time: 0 seconds)

Zmienne w interfejsie

Interfejsów można używać do wprowadzania w wielu klasach tych samych stałych. Wystarczy w interfejsie zadeklarować i zainicjować zmienne. Klasa implementująca taki interfejs będzie w swoim zasięgu widziała wszystkie te zmienne jako stałe (przypomina to tworzenie stałych dyrektywą *#define* lub deklaracją *const* w języku C lub C++). Jeśli interfejs nie zawiera żadnych metod, wtedy implementująca go klasa nie musi deklarować żadnych dodatkowych metod. Zmienne występujące w interfejsie są widoczne przez klasę korzystającą z interfejsu jako *final*. Kolejny przykład używa interfejsu do zaimplementowania automatu podejmującego decyzję.

```
package p7;
interface SharedConstants {
    int NO = 0;
    int YES = 1;
    int MAYBE = 2;
    int LATER = 3;
    int SOON = 4;
    int NEVER = 5;
}
```

```
package p7;
import java.util.Random;
public class Question implements SharedConstants {
    Random rand = new Random();
    int ask() {
        int prob = (int) (100*rand.nextDouble());
        if(prob < 30) return NO;
        else
            if(prob < 60) return YES;
        else
            if(prob < 70) return MAYBE;
        else
            if(prob < 85) return SOON;
        else
            return NEVER;
    }
}
```

```
package p7;
public class AskMe implements SharedConstants{
    static void answer(int result){
        switch(result){
            case NO:
                System.out.println("Nie");
                break;
            case YES:
                System.out.println("Tak");
                break;
            case MAYBE:
                System.out.println("Może");
                break;
            case LATER:
                System.out.println("Później");
                break;
            case SOON:
                System.out.println("Wkrótce");
                break;
            case NEVER:
                System.out.println("Nigdy");
                break;
        }
    }
}
```

```
package p7;

public class P7 {

    public static void main(String[] args) {
        Question q = new Question();
        AskMe.answer(q.ask());
        AskMe.answer(q.ask());
        AskMe.answer(q.ask());
        AskMe.answer(q.ask());
    }
}
```

Wyjście - P7 (run) ×

run:
Tak
Wkrótce
Tak
Wkrótce
BUILD SUCCESSFUL (total time: 0 seconds)

Wyjście - P7 (run) ×

run:
Wkrótce
Nie
Tak
Nigdy
BUILD SUCCESSFUL (total time: 0 seconds)

Wyjaśnij pojęcie abstrakcji w programowaniu obiektowym

Każdy obiekt w systemie służy jako model abstrakcyjnego „wykonawcy”, który może wykonywać pracę, opisywać i zmieniać swój stan oraz komunikować się z innymi obiektami w systemie bez ujawniania, w jaki sposób zaimplementowano dane cechy. Procesy, funkcje lub metody mogą być również abstrahowane, a kiedy tak się dzieje, konieczne są rozmaite techniki rozszerzania abstrakcji.

Klasy abstrakcyjne

Przypuśćmy, że klasa podstawowa (bazowa) Pracownik opisująca pracownika dostarcza swoim podklasom potomnym chronione zmienne oraz metody wirtualne, które mogą zostać przysłonięte w poszczególnych podklasach. Wszystko byłoby w porządku, gdyby nie możliwość bezpośredniego utworzenia instancji klasy podstawowej.

```
// Klasa ma służyć jako wzorzec  
// Do konstrukcji podklas  
Pracownik p = new Pracownik();
```

W języku Java istnieje możliwość ograniczenia bezpośredniego tworzenia instancji klasy (obiektu) przez dodanie słowa kluczowego *abstract*.

```
public abstract class Pracownik { //...  
}
```

Pracownik is abstract; cannot be instantiated

(Alt-Enter shows hints)

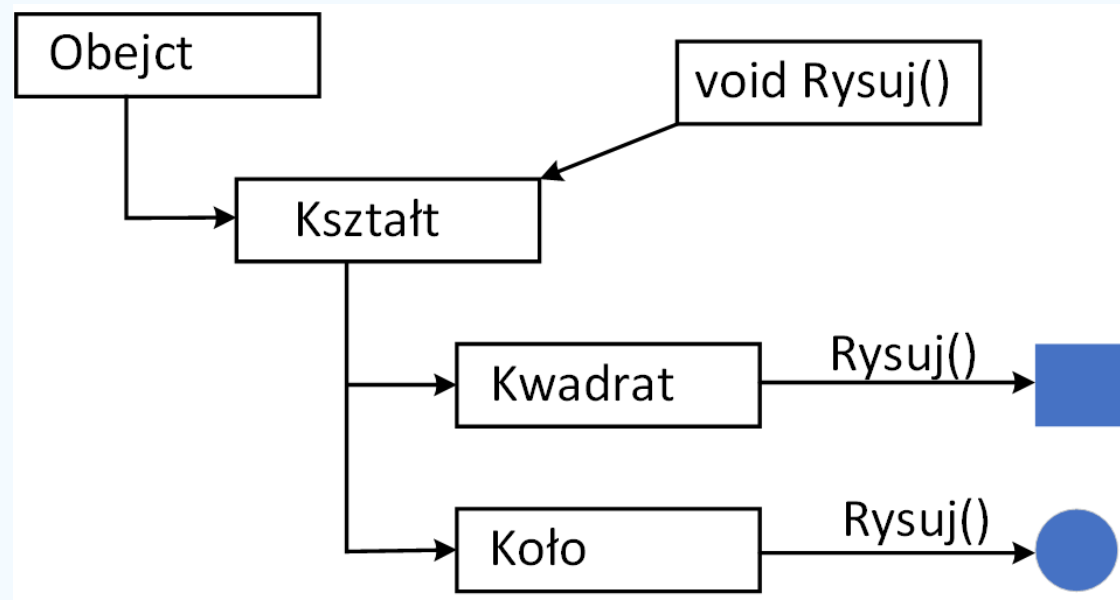
```
(String[] args) {
```

```
    Pracownik p = new Pracownik();
```

Metody abstrakcyjne

Po zdeklarowaniu klasy abstrakcyjnej można w niej deklarować abstrakcyjne składowe. Metody abstrakcyjne można stosować wszędzie tam, gdzie trzeba zdefiniować metodę, która nie jest implementowana w klasie abstrakcyjnej. W ten sposób zostaje wymuszony polimorfizm w klasach potomnych, gdzie muszą zostać zaimplementowane w sposób realizujący zadania w klasie potomnej.

Rozpatrzmy przykład klasy abstrakcyjnej Kształt, w której znajdują się różne figury geometryczne.



```
package p2;
public abstract class Ksztalt {
    protected String nazwa;
    public Ksztalt() { nazwa = "nieznany"; }
    public Ksztalt(String nazwa) {
        this.nazwa = nazwa;
    }
    public void Rysuj() {
        System.out.println("Ksztalt.Rysuj()");
    }
}
```

```
package p2;
public class Kolo extends Ksztalt {
    public Kolo() { }
    public Kolo(String nazwa) { super(nazwa); }
}
```

```
package p2;
public class Kwadrat extends Ksztalt{
    public Kwadrat() { }
    public Kwadrat(String nazwa) { super.nazwa=nazwa; }
    @Override
    public void Rysuj()
    {
        System.out.printf("Rysuję %s kwadrat \n", nazwa);
    }
}
```

```
package p2;
public class P2 {
    public static void main(String[] args) {
        Kwadrat kw = new Kwadrat("niebieski");
        kw.Rysuj();
        Kolo ko = new Kolo("czerwone");
        ko.Rysuj();
    }
}
```

Wyjście - P2 (run) ×

run:
Rysuję niebieski kwadrat
Kształt.Rysuj()
BUILD SUCCESSFUL (total time: 0 seconds)

Działanie programu

Zauważmy, że klasa *Kształt* definiuje metodę *Rysuj()* jako wirtualną. Podklasy mogą, ale nie muszą przeddefiniować metody wirtualnej. Jeśli metoda ma być przesłonięta (tak jak w przypadku klasy *Kwadrat*), to trzeba użyć słowa kluczowego *override*. Łatwo zauważyć, że podklasy nie muszą przysłaniać metod wirtualnych (tak jak robi to klasa *Koło*). Dlatego jeśli utworzymy instancje obiektów klas *Kwadrat* i *Koło*, okaże się że, obiekt klasy *Kwadrat* potrafi poprawnie wypisać informacje, podczas, gdy obiekt klasy *Koło* nie potrafi tego zrobić.

Abstrakcyjne metody mogą być realizowane tylko w klasach abstrakcyjnych. Jeśli spróbujemy zrobić inaczej, otrzymamy błąd kompilacji. W związku z powyższym trzeba zadeklarować metodę *Rysuj()* w klasie *Koło*. Jeśli tego nie wykonamy, *Koło* zostanie za abstrakcyjny typ, którego obiektów tworzyć nie można i należałoby go oznaczyć słowem kluczowym *abstract*. W naszej implementacji jest to po prostu błąd i po poprawnej modyfikacji program wygląda następująco:

```
package p3;
public abstract class Ksztalt {
    protected String nazwa;
    public Ksztalt() { nazwa = "nieznany"; }
    public Ksztalt(String nazwa){
        this.nazwa = nazwa;
    }
    public abstract void Rysuj();
}
```

```
package p3;
public class Kolo extends Kształt{
[- public Kolo() { }
[-     public Kolo(String nazwa) { super(nazwa); }
    @Override
[- public void Rysuj() {
    |     System.out.printf("Rysuję %s koło \n", nazwa);
    | }
    }
}
```

```
package p3;
public class Kwadrat extends Kształt{
    public Kwadrat() { }
    public Kwadrat(String nazwa) {super(nazwa); }
    @Override
    public void Rysuj() {
        System.out.printf("Rysuję %s kwadrat \n", nazwa);
    }
}
```

```
package p3;
public class P3 {
    public static void main(String[] args) {
        Kształt[] ks = {new Kwadrat(), new Kolo(),
            new Kwadrat("czewony"), new Kolo("żółty"),
            new Kwadrat("niebieski") };

        for (int i = 0; i < ks.length; i++)
            ks[i].Rysuj();
    }
}
```

Wyjście - P3 (run) ×



run:



Rysuję nieznany kwadrat



Rysuję nieznany koło



Rysuję czewony kwadrat

Rysuję żółty koło

Rysuję niebieski kwadrat

BUILD SUCCESSFUL (total time: 0 seconds)

Wyjaśnij pojęcie hermetyzacji w programowaniu obiektowym

Czyli ukrywanie implementacji, enkapsulacja. Zapewnia, że obiekt nie może zmieniać stanu wewnętrznego innych obiektów w nieoczekiwany sposób. Tylko własne metody obiektu są uprawnione do zmiany jego stanu. Każdy typ obiektu prezentuje innym obiektom swój interfejs, który określa dopuszczalne metody współpracy. Pewne języki osłabiają to założenie, dopuszczając pewien poziom bezpośredniego (kontrolowanego) dostępu do „wnętrza” obiektu. Ograniczają w ten sposób poziom abstrakcji. Przykładowo w niektórych kompilatorach języka C++ istnieje możliwość tymczasowego wyłączenia mechanizmu enkapsulacji; otwiera to dostęp do wszystkich pól i metod prywatnych, ułatwiając programistom pracę nad pośrednimi etapami tworzenia kodu i znajdowaniem błędów.

Enkapsulacja, czyli hermetyzacja

Koncepcja hermetyzacji wyraża myśl, że wewnętrzne dane obiektu nie powinny być bezpośrednio dostępne z instancji obiektu. Jeżeli użytkownik obiektu ma zamiar zmienić jego stan, to może to zrobić pośrednio, wykorzystując w tym celu metody udostępniające i modyfikujące. W języku Java hermetyzacja jest ustalona na poziomie synkratycznym za pomocą słów kluczowych *public*, *private* i *protected*. Rozważmy klasę Pracownik. Jeżeli mają być interakcje zewnętrzne z prywatnymi zmiennymi, to zgodnie z tradycyjnym podejściem należy zdefiniować metodę udostępniającą (get) oraz metodę modyfikującą (set). Poza tradycyjnymi metodami udostępniającymi i modyfikującymi, klasy posiadają także właściwości (ang. *properties*) np..

```
package p4;
public class Pracownik {
    private int id;
    private String nazwisko;
    private double placa;
    // Deklaracja pól klasy
    public void setId(int id)
    // Właściwość klasy
    { this.id =id;
      }
    public int getId(){
        return id;
    }
    public String getNazwiako() { return nazwisko; }
    // Metoda udostępniająca zmienną
    public Pracownik() { }
        //Przeciążenie konstruktora domyślnego
    public Pracownik(int id, String nazwisko,double placa){
        //Konstruktor publiczny
        this.id = id;
        this.nazwisko = nazwisko;
        this.placa = placa;
    }
}
```

```
@Override
    public String toString()
    {
        //Przeciążenie metody ToString()
        String s = String.format
        ("id: %d nazwisko: %s płaca: %g", id, nazwisko, placa);
        return s;
    }
}
```

Wyjście - P4 (run) ×



run:



Id: 1 Nazwisko: Kowalski



id: 1 nazwisko: Kowalski płaca: 1200,00



Id: 81 Nazwisko: Kowalskiid: 81 nazwisko: Kowalski płaca: 1200,00

BUILD SUCCESSFUL (total time: 0 seconds)

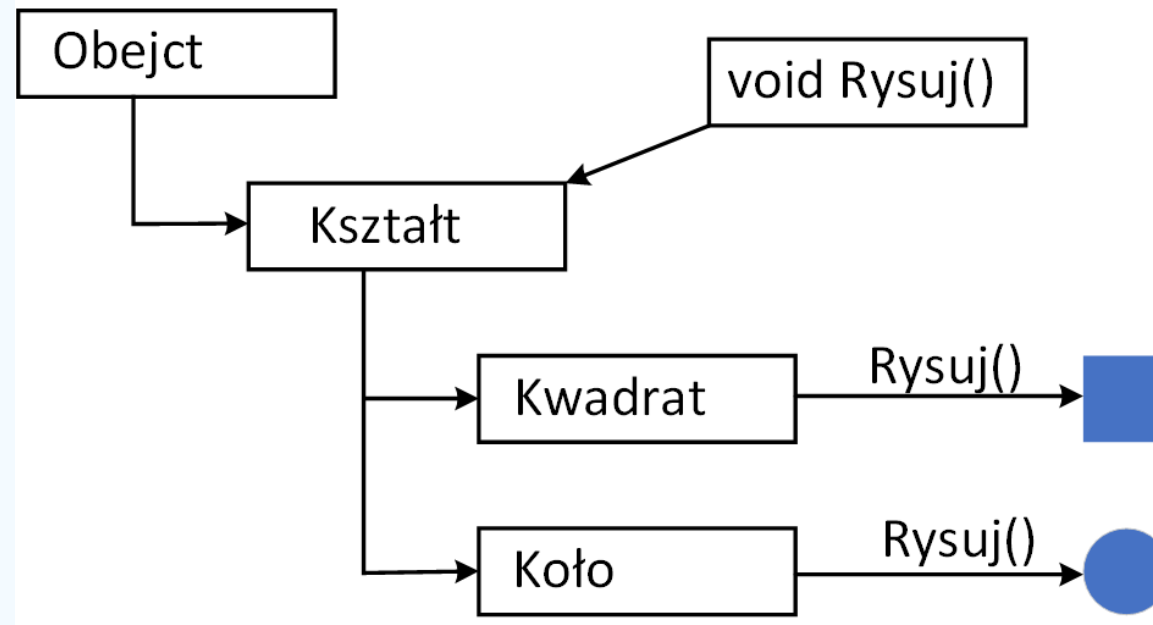
Wyjaśnij pojęcie polimorfizmu w programowaniu obiektowym

Polimorfizm oznacza zdolność przyjmowania różnych postaci. W programowaniu obiektowym wiele postaci mogą przyjmować zmienne, encje lub element struktury danych, gdyż mają zdolność związania się z obiektami różnych typów. Wszystko jest kontrolowane przez statyczne deklaracje.

Polimorfizm to zdolność języka do traktowania spokrewnionych obiektów w taki sam sposób. Rozróżniamy dwa rodzaje polimorfizmu: klasyczny i ad hoc. W klasycznym polimorfizmie możliwe jest zdefiniowanie w klasie podstawowej (klasie rodzica) zbioru składowych, które można przysłaniać (ang. override) w podklasie. Kiedy metoda przysłania zachowanie zdefiniowane w klasie podstawowej, to właściwie nowa ustala, jak odpowie na taki sam komunikat.

Polimorfizm

Powróćmy do programu realizującego hierarchię figur geometrycznych. Zauważmy, że w klasie *Kształt* jest zdefiniowana metoda *Rysuj()*, która nie pobiera żadnych parametrów i niczego nie zwraca. Ponieważ każda figura wypisuje inny komunikat, poszczególne podklasy *Kwadrat* i *Koło* powinny zinterpretować metodę *Rysuj()* po swojemu (patrz rysunek).



Polimorfizm

W klasycznym polimorfizmie klasa podstawowa ma możliwość wymuszania określonego zachowania na wszystkich swoich podklasach. Na rysunku widać, że każdy obiekt wywodzący się z klasy *Kształt* potrafi się narysować. To zaleta języka obiektowego, ponieważ polimorfizm pozwala uniknąć tworzenia niepotrzebnych metod przeprowadzających niepotrzebne operacje np. *RysujKwadrat()*, *RysujKolo()* itp..

Polimorfizm ad hoc pozwala, aby obiekty, które nie są związane klasyczną dziedzicznością, były traktowane w podobny sposób, pod warunkiem, że każdy obiekt ma metodę o dokładnie takim samym nagłówku (czyli nazwie, liście parametrów oraz zwracanym typie). Języki które posługują się polimorfizmem ad hoc, posługują się techniką tak zwaną późnym wiązaniem służącą do wykrywania typu danych podczas wykonywania programu. Od wyniku rozpoznania typu zależy, która metoda zostanie wywołana.

```
package p3;
public class P3 {
    public static void main(String[] args) {
        Kształt[] ks = {new Kwadrat(), new Kolo(),
            new Kwadrat("czewony"), new Kolo("żółty"),
            new Kwadrat("niebieski") };

        for (int i = 0; i < ks.length; i++)
            ks[i].Rysuj();
    }
}
```

Wyjście - P3 (run) ×



run:



Rysuję nieznany kwadrat



Rysuję nieznany koło



Rysuję czewony kwadrat

Rysuję żółty koło

Rysuję niebieski kwadrat

BUILD SUCCESSFUL (total time: 0 seconds)

Przedstaw zastosowanie dziedziczenia w programowaniu obiektowym

Porządkuje i wspomaga polimorfizm i enkapsulację dzięki umożliwieniu definiowania i tworzenia specjalizowanych obiektów na podstawie bardziej ogólnych. Dla obiektów specjalizowanych nie trzeba redefiniować całej funkcjonalności, lecz tylko tę, której nie ma obiekt ogólniejszy. W typowym przypadku powstają grupy obiektów zwane *klasami*, oraz grupy klas zwane *drzewami*. Odzwierciedlają one wspólne cechy obiektów.

Klasycznie dziedziczenie to rozwiązanie zależności między klasą podstawową i klasą potomną. Przedstawmy sposób dziedziczenia w języku Java na przykładzie.

```
package p5;
public class PunktX {
    protected double x; //{ get; set; }
    // Deklaracja pola
    public PunktX() {}
    // Przeciążenie konstruktora domyślnego
    public PunktX(double x) {
        //Konstruktor publiczny
        this.x = x;
    }
    @Override
    public String toString() {
        //Przeciążenie metody ToString()
        String s = "( " + x + " )";
        return s;
    }
}
```

```
package p5;
public class PunktXY extends PunktX {
    protected double y; //{ get; set; }
    //Pole klasy
    public PunktXY() {}
    //Przeciążenie konstruktora domyślnego
    public PunktXY(double x, double y) {
        //Konstruktor publiczny
        this.x= x;
        this.y = y;
    }
    @Override
    public String toString() {
        //Przeciążenie metody ToString()
        String s = "( " + x + " ; " + y + " )";
        return s;
    }
}
```

```
package p5;
public class PunktXYZ extends PunktXY{
    private double z; //{ get; set; }
    //Pole klasy
    public PunktXYZ() {}
    //Przciążenie konstruktora domyślnego
    public PunktXYZ(double x, double y, double z) {
        //Konstruktor publiczny
        this.x = x;
        this.y = y;
        this.z = z;
    }
    @Override
    public String toString() {
        // Przeciążenie metody ToString()
        String s = "( " + x + " ; " + y + " ; " + z + " )";
        return s;
    }
}
```

```
package p5;
public class P5 {
    public static void main(String[] args) {
        PunktX d1 = new PunktX(12.1);
        //Utworzenie obiektu klasy bazowej PunktX
        System.out.println(d1);
        PunktXYZ c1 = new PunktXYZ(3.4, 3.6, 8.9);
        //Utworzenie obiektu klasy potomnej PunktXYZ
        System.out.println(c1);
        PunktXY c2 = new PunktXY(-1.2, -2.3);
        //Utworzenie obiektu klasy potomnej PunktXY
        System.out.println(c2);
        PunktX c3 = new PunktX(10.5);
        //Utworzenie obiektu klasy bazowej PunktX
        System.out.println(c3);
    }
}
```



run:



(12.1)



(3.4 ; 3.6 ; 8.9)



(-1.2 ; -2.3)



(10.5)

BUILD SUCCESSFUL (total time: 0 seconds)

Przedstaw zastosowania przeciążenia metod i operatorów w programowaniu obiektowym

Języki zorientowane obiektowo pozwalają na przeciążanie różnych składowych. Jeżeli klasa zawiera wiele identycznie nazwanych metod różniących się liczbą lub typem parametrów, to taką metodę nazywamy przeciążoną. Klasa Pracownik ma przeciążony konstruktor klasy, ponieważ istnieją dwie jego definicje różniące się zestawem parametrów.

W momencie przeciążonej metody Java potrzebuje typów i (lub) liczby argumentów do określenia, którą wersję przeciążonej metody należy wywołać. Choć przeciążone metody mogą zwracać wartości różnych typów, znaczenie ma liczba i typ argumentów. Z tego powodu przeciążone metody muszą różnić się parametrami, a nie tylko typem zwracanej wartości. Java w momencie napotykania wywoływania przeciążonej metody szuka wersji najlepiej pasującej do przekazanych argumentów i wykonuje jej kod.

Przeciążenie metod

```
package p9;
public class Przeciazona {
    void test() {
        System.out.println("Brak parametrów");
    }
    void test(int a) {
        System.out.println("a: " + a);
    }
    int test(int a, int b) {
        return a + b;
    }
    double test(double a, double b) {
        return a + b;
    }
}
```

```
package p9;
public class P9 {
    public static void main(String[] args) {
        Przeciazona p = new Przeciazona();
        p.test();
        p.test(9);
        int t = p.test(1, 2);
        System.out.println("a + b = " + t);
        double z = p.test(1.0, 2.0);
        System.out.println("a + b = " + z);
    }
}
```



run:



Brak parametrów



a: 9



a + b = 3

a + b = 3.0

BUILD SUCCESSFUL (total time: 0 seconds)

Metoda *test()* jest przeciążona cztery razy. Pierwsza wersja nie przyjmuje żadnych parametrów, druga przyjmuje jeden parametr, trzecia i czwarta przyjmuje dwa parametry różniące się typami. Dodatkowo czwarta metoda zwraca typ *double*. Ale nie wpływa to w żaden sposób na wybór odpowiedniej metody.

Przeciążenie metod to element polimorfizmu, gdyż to jedyne rozwiązanie zapewniające spełnienie w Javie paradygmatu „jeden interfejs, wiele metod”. W języku programowania nieobsługującym przeciążenia metod trzeba by nadać każdej metodzie unikatową nazwę. W języku C funkcja **abs()** dotyczy liczb całkowitych, *labs()* liczby long, natomiast *fabs()* liczb zmiennoprzecinkowych. Jak łatwo zauważyć nie jest to rozwiązanie wygodne, a przy większych projektach staje się problemem. W Javie przedstawiony problem nie występuje – istnieje tylko jedna nazwa dla metody zwracającej bezwzględną i jest to metoda *abs()* z klasy Math. Klasa zawiera wiele przeciążonych metod przeciążonych tej metody obsługujące wszystkie podstawowe typy numeryczne.

Przeciążenie konstruktorów

Przeciążenie nie dotyczy tylko metod i może być stosowane dla konstruktorów. W zasadzie w większości programów w Javy przeciążone konstruktory to raczej norma niż wyjątek.

```
public class Pracownik {  
    private int id;  
    private String nazwisko;  
    private double placa;  
    // Deklaracja pól klasy  
    public Pracownik() {  
        this.id = 0;  
        this.nazwisko = "";  
        this.placa = 0.0;  
    }  
    //Przeciążenie konstruktora domyślnego  
    public Pracownik(int id, String nazwisko, double placa){  
        this.id = id;  
        this.nazwisko = nazwisko;  
        this.placa = placa;  
    }  
}
```

Przeciążenie metod i operatorów

Przykładem przeciążonego operatora w języku Java jest operator dodawania, który może oznaczać dodawanie liczb albo łączenie łańcuchów znaków. Język C# dopuszcza ponowne przeciążenie operatora dodawania przez programistę. Takie przeciążenie nie jest dopuszczalne w języku Java. W języku Java w celu stworzenia klasy liczb zespolonych trzeba użyć metod implementujących operacje arytmetyczne wykonywane na tych liczbach.